Department of Electrical Engineering

ASIC/FPGA Chip Design

Laboratory Manual

ASIC/FPGA Chip Design Laboratory Manual

Dr. Mahdi Shabany

Spring 2025

Contents

Lab 5 - Introduction to Zynq & getting started with PS

1.1 Objective

In this lab, you become familiar with the Zynq SoC and some minimal capabilities and interfaces of the versatile chip. These include initialization of PS and UART for Debugging, MIOs and EMIOs of PS and control them (like LEDs), PS/PL connections, PS and PL data interaction through GP port and AXI GPIO and testing a module by this way.

1.2 Introduction

To realize your descripted digital circuit using Hardware Description Language (HDL), we have used FPGAs in the previous labs.

FPGAs are used to create any specific hardware you want, there is almost no built-in fixed hardware on the chip, you need to create the hardware dedicated to performing your particular application. Because the designed hardware is customized for the specific task, you can benefit from its high performance and parallelism. The drawback might be high power consumption.

Microcontrollers on the other hand, are a set of fixed circuits, each designed to be able to carry out some basic widely used tasks. By programming the microcontroller, you are actually configuring the general-purpose chip to execute your operations one by one.

Designers have long known that general-purpose processors are flexible, while single-purpose processors are fast. Most applications include both general-purpose needing tasks and single-purpose tasks. To address the demand of a chip able to execute both applications, in March 2011, Xilinx introduced the Zynq-7000 family, which integrates the software programmability of an ARM®-based processor (PS for Processing System) with the hardware programmability of an FPGA (PL for Programmable Logic). In addition to its dual-core ARM processor, PS includes various other components such as memory, ports, protocols, and implemented peripherals.

In other words, we have two types of algorithms. The first group is static algorithms which performs a fixed set of operations on its input data (for example, computing the inverse of an input matrix). Most signal processing and image processing algorithms fall into this category. These operations must run at maximum speed and with minimal latency. Because the algorithm is static, it's possible to implement a dedicated hardware circuit optimized for that algorithm; accordingly, we implement such circuits in the FPGA.

The second group consists of adaptive algorithms those filled with various, unpredictable conditions (in plain English, lots of if-else statements). Due to resource constraints and other limitations, it's not practical to build custom hardware for these algorithms. Instead, we execute them on processors or microcontrollers.

Most of our projects include both static and adaptive algorithm components. The Zynq chip satisfies this requirement. Zynq is a SoC (System on Chip), meaning an entire system is contained within a single chip.

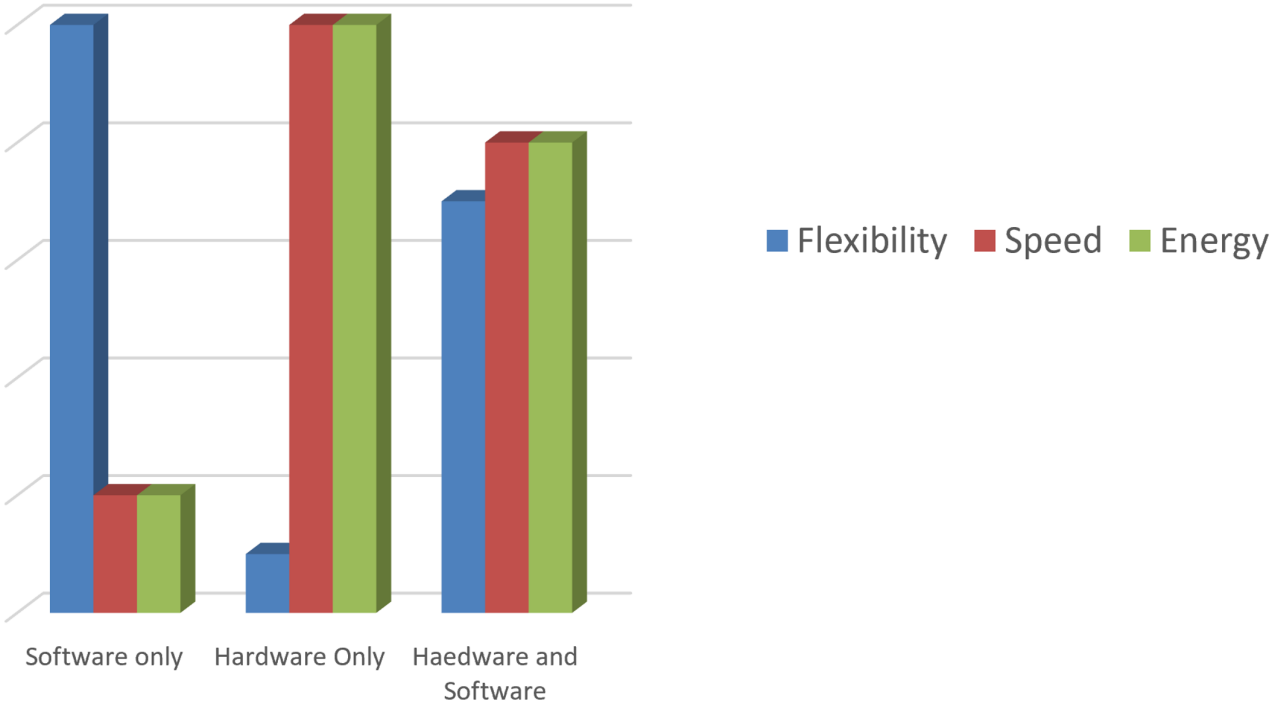

Figure 1.1: Comparison of hardware-only systems (FPGA), software-only systems (microcontroller), and their integration, in terms of flexibility, speed, and power consumption

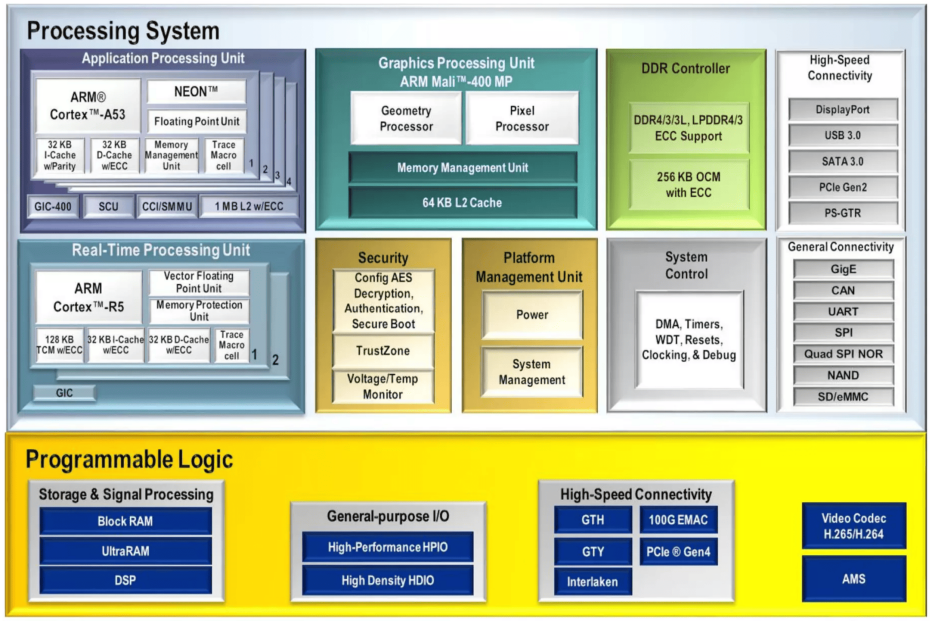

Figure 1.2: Internal functional blocks of the Zynq-UltraScale+ chip

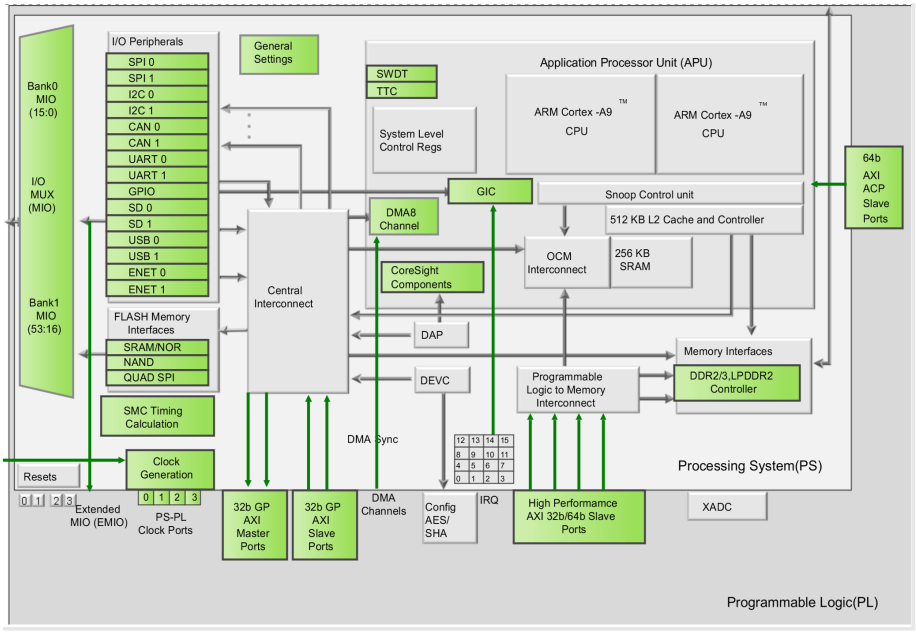

Figure 1.3: Cross-sectional view inside the Zynq-7000 chip

For more details on the Zynq chip, please refer to the Zynq 7000 Reference Manual (UG585).

	Device Name	Z-7007S	Z-7012S	Z-7014S	Z-7010	Z-7015	Z-7020	Z-7030	Z-7035	Z-7045	Z-7100
	Part Number	XC7Z007S	XC7Z012S	XC7Z014S	XC7Z010	XC7Z015	XC7Z020	XC7Z030	XC7Z035	XC7Z045	XC7Z100
Programmable Logic	Xilinx 7 Series Programmable Logic Equivalent	Artix®-7 FPGA	Artix-7 FPGA	Artix-7 FPGA	Artix-7 FPGA	Artix-7 FPGA	Artix-7 FPGA	Kintex®-7 FPGA	Kintex-7 FPGA	Kintex-7 FPGA	Kintex-7 FPGA
	Programmable Logic Cells	23K	55K	65K	28K	74K	85K	125K	275K	350K	444K
	Look-Up Tables (LUTs)	14,400	34,400	40,600	17,600	46,200	53,200	78,600	171,900	218,600	277,400
	Flip-Flops	28,800	68,800	81,200	35,200	92,400	106,400	157,200	343,800	437,200	554,800
	Block RAM (# 36 Kb Blocks)	1.8 Mb (50)	2.5 Mb (72)	3.8 Mb (107)	2.1 Mb (60)	3.3 Mb (95)	4.9 Mb (140)	9.3 Mb (265)	17.6 Mb (500)	19.2 Mb (545)	26.5 Mb (755)
	DSP Slices (18x25 MACCs)	66	120	170	80	160	220	400	900	900	2,020
	Peak DSP Performance (Symmetric FIR)	73 GMACs	131 GMACs	187 GMACs	100 GMACs	200 GMACs	276 GMACs	593 GMACs	1,334 GMACs	1,334 GMACs	2,622 GMACs
	PCI Express (Root Complex or Endpoint)[3]		Gen2 x4			Gen2 x4		Gen2 x4	Gen2 x8	Gen2 x8	Gen2 x8
	Analog Mixed Signal (AMS) / XADC	2x 12 bit, MSPS ADCs with up to 17 Differential Inputs									
	Security[2]	AES and SHA 256b for Boot Code and Programmable Logic Configuration, Decryption, and Authentication									

Figure 1.4: Comparison of Configurations Across Different Zynq 7000 Models

	Device Name	Z-7007S	Z-7012S	Z-7014S	Z-7010	Z-7015	Z-7020	Z-7030	Z-7035	Z-7045	Z-7100
	Part Number	XC7Z007S	XC7Z012S	XC7Z014S	XC7Z010	XC7Z015	XC7Z020	XC7Z030	XC7Z035	XC7Z045	XC7Z100
Processing System	Processor Core	Single-core ARM Cortex-A9 MPCore™ with CoreSight™			Dual-core ARM Cortex-A9 MPCore™ with CoreSight™						
	Processor Extensions	NEON™ & Single / Double Precision Floating Point for each processor									
	Maximum Frequency	667 MHz (-1); 766 MHz (-2)			667 MHz (-1); 766 MHz (-2); 866 MHz (-3)			667 MHz (-1); 800 MHz (-2); 1 GHz (-3)			667 MHz (-1) 800 MHz (-2)
	L1 Cache	32 KB Instruction, 32 KB data per processor									
	L2 Cache	512 KB									
	On-Chip Memory	256 KB									
	External Memory Support[1]	DDR3, DDR3L, DDR2, LPDDR2									
	External Static Memory Support[1]	2x Quad-SPI, NAND, NOR									
	DMA Channels	8 (4 dedicated to Programmable Logic)									
	Peripherals[1]	2x UART, 2x CAN 2.0B, 2x I2C, 2x SPI, 4x 32b GPIO									
	Peripherals w/ built-in DMA[1]	2x USB 2.0 (OTG), 2x Tri-mode Gigabit Ethernet, 2x SD/SDIO									
	Security[2]	RSA Authentication, and AES and SHA 256-bit Decryption and Authentication for Secure Boot									
Processing System to Programmable Logic Interface Ports (Primary Interfaces & Interrupts Only)		2x AXI 32b Master 2x AXI 32-bit Slave 4x AXI 64-bit/32-bit Memory AXI 64-bit ACP 16 Interrupts									

Figure 1.5: Excerpt from UG585 features and specifications of the PL and PS for different Zynq models

Thanks to its unique capabilities, Zynq has secured a strong position in both the Iranian and global industries. Its high production volumes have driven its price down, making it more cost-effective than some weaker chips. For example, the table below compares the features and prices of a Zynq device versus Xilinx's Artix FPGA series (note that the FPGA fabric inside this Zynq is itself an Artix):

	Artix	**Zynq**	**Artix**
	XC7A75T	**XC7z020**	**XC10075T**
Logic Cells	75k	85k	101k
Block RAM	210	280	270
DSP48	180	220	240
GTx	8	0	8
Price	116$	104$	137$

Figure 1.6: Comparison of features and prices for Zynq versus Artix devices

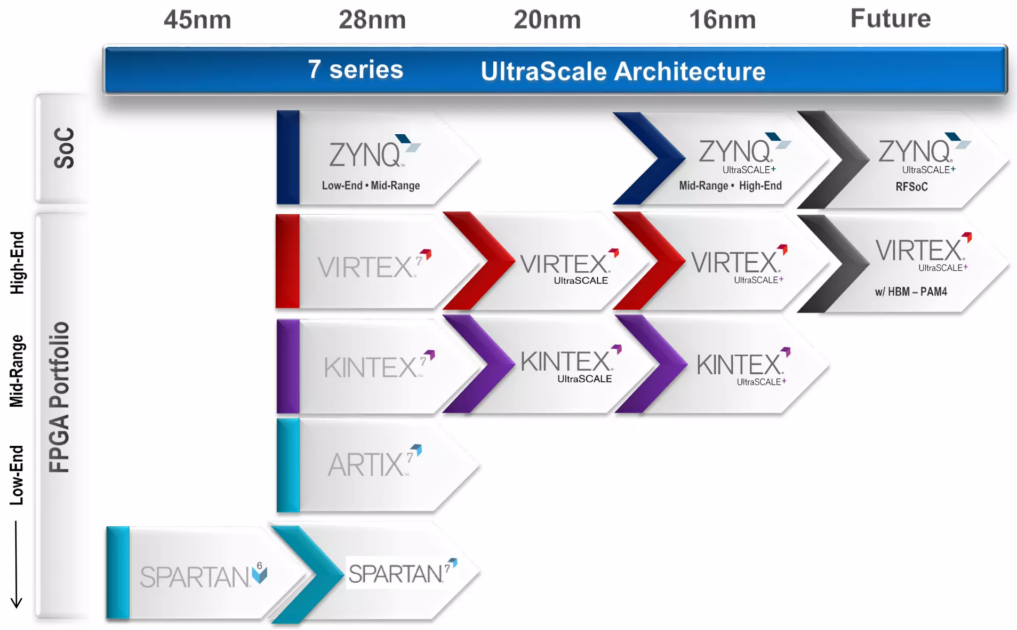

Figure 1.7: Zynq fabrication technology compared with other Xilinx chips

For more conceptual diagrams of the Zynq architecture, check this link out.

To gain a better understanding of these concepts, it is recommended that you watch this video.

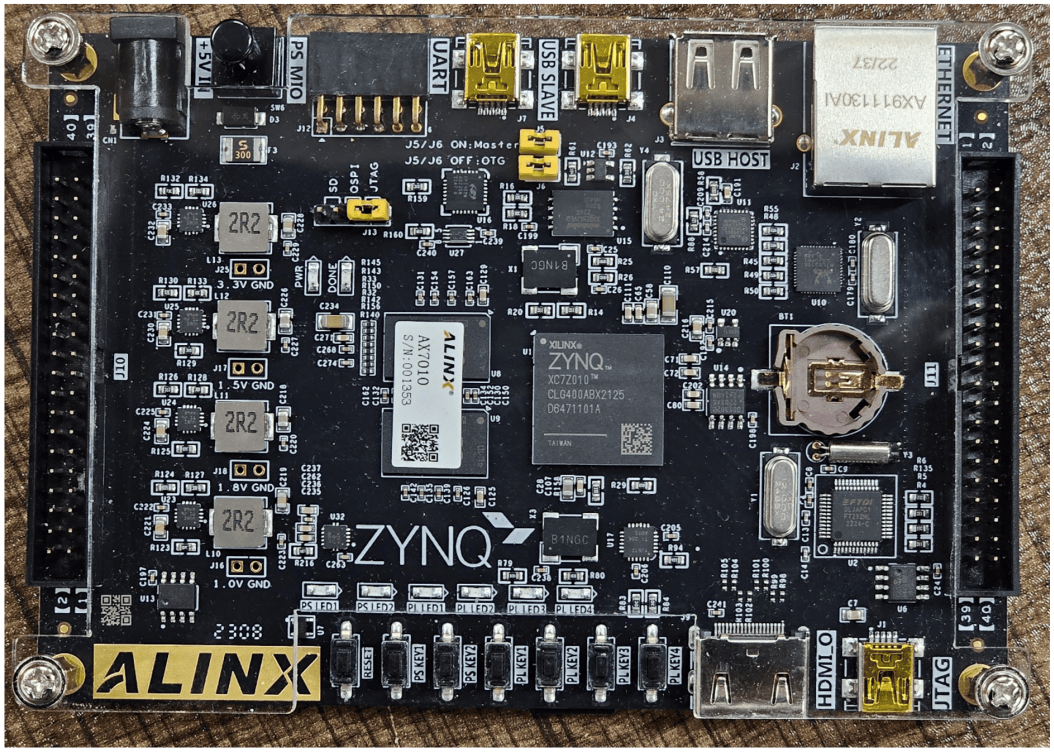

Figure 1.8: The picture of the Zynq board (AX7010), you will work with in the laboratory

In this lab, you will learn the PS/PL connections by verifying some modules. These verification methods will turn out to be useful in the future. Before you begin the lab, please read the following document completely. this should answer many of your questions. If you have any other questions or problems - ask!

And you need to watch videos 8, 10.a and 10.b from the Zynq tutorial series to learn how to initialize the PS and its MIOs and to connect it to the PL through the GP port and AXI GPIO.

It is also highly recommended to watch this video to learn how to get started with the PS and initialize its UART.

1.3 Instructions

1.3.1 Initializing the PS UART

First, you are to configure the PS IP Core in Vivado. Just initialize the UART of the PS for debugging. Find the correct pins of UART (from 54 pins of PS MIO) for this board in its datasheet. Now run the template "Hello World" project in the SDK and test the PS's UART with a serial terminal app (like Putty or SDK terminal).

1.3.2 Terminal-Controlled PS-LED Blinking via GPIO MIO

Now, get a number from the user in the terminal (1 to 10) and write a program to blink one of the PS LEDs at a frequency according to the user's number. Find the PS's LED in the board's datasheet or schematic (for this part, you should have enabled the PS's GPIO MIOs in Vivado).

1.3.3 Bonus - PL-LED Blinking via GPIO EMIO

For the bonus mark, do the last task on one of the PL side LEDs by the PS's GPIO EMIO (NOT GP Port and AXI GPIO).

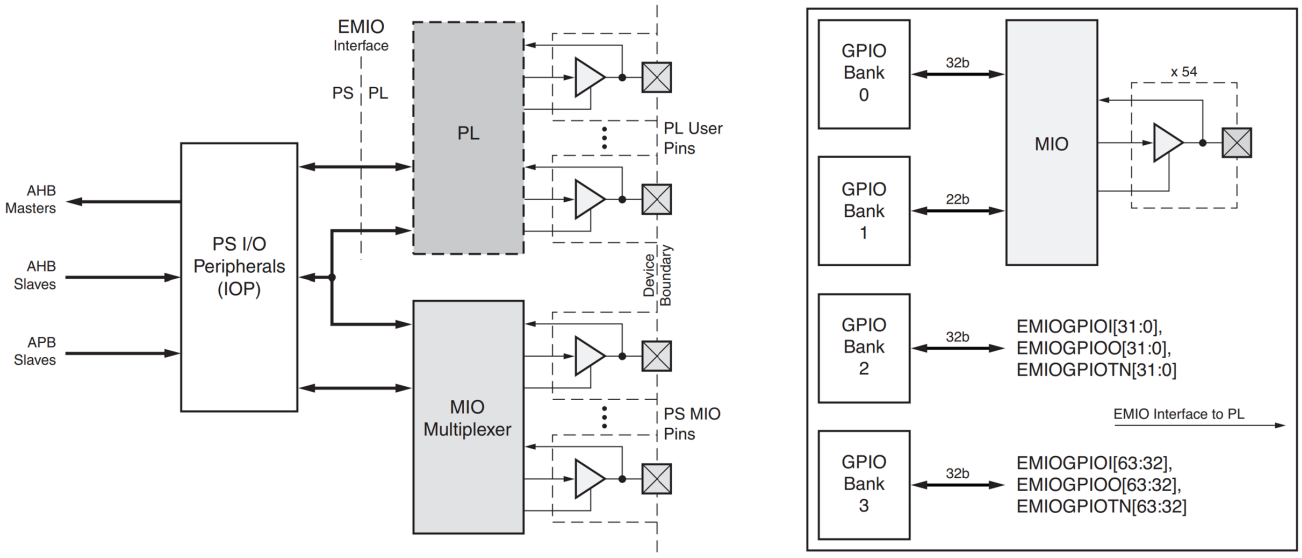

Figure 1.9: Block Diagram of PS-PL Communication via GPIO EMIO

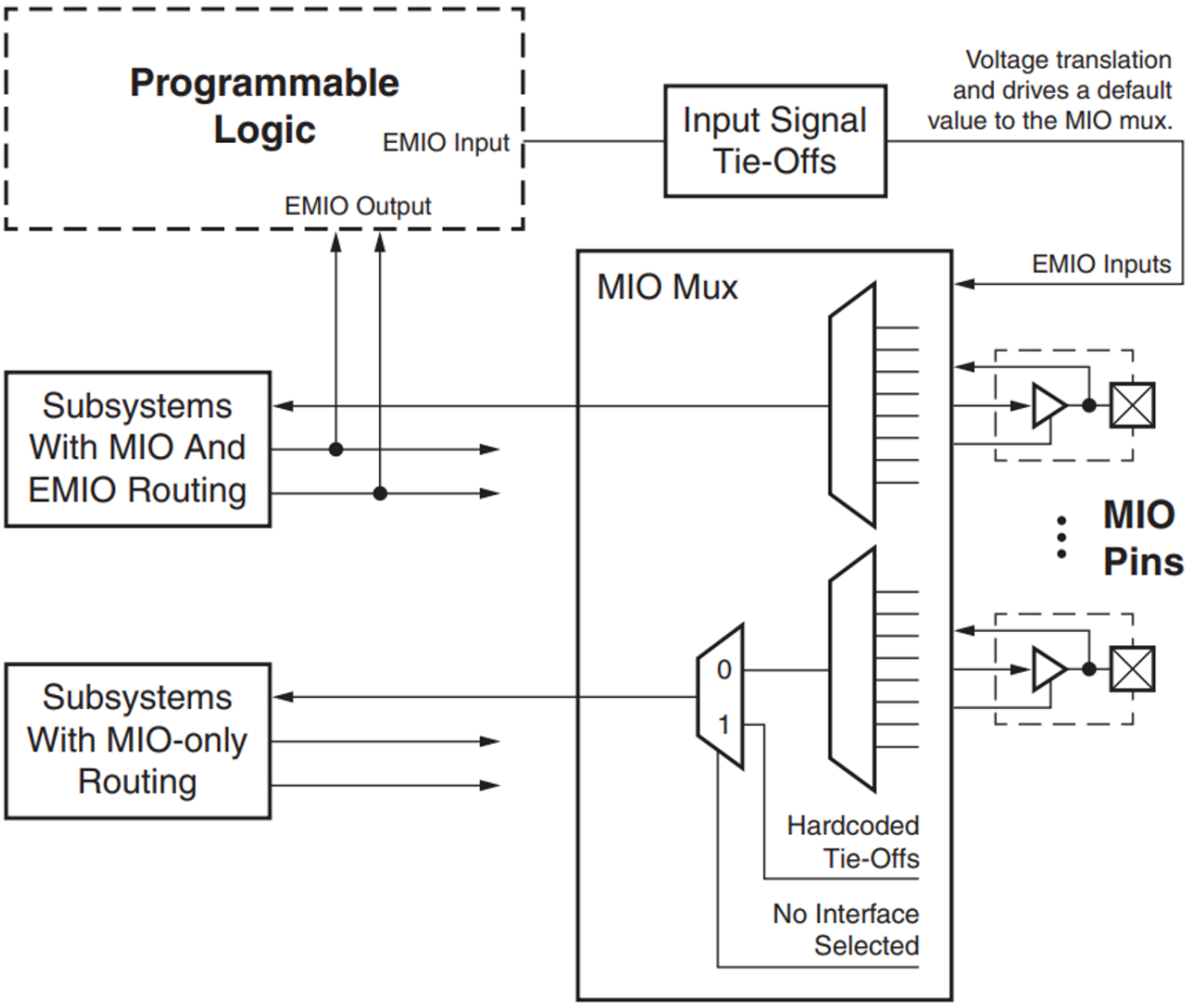

Figure 1.10: Some pictures of the architecture of the PS's GPIO-MIO, EMIO. (For more information, you can see the Zynq-7000 Reference Manual (UG585) or listen to the TA's explanation in the Lab session!)

1.3.4 ALU Functionality Verification in PL via PS Inputs

Now you need to verify a simple 32-bit ALU using PS. You should first describe the ALU module to be implemented on the PL, then connect the PL to the ARM processor (PS), ultimately create a software program to test the module with 5,000,000 randomly created test vectors.

Whenever you create a module, you need to test it. First off, you simulate your module and test it via a testbench. The testbench as you know, creates some inputs, and feeds them into your module and gets the output of it to be checked. But when you implement it on a FPGA, you cannot use testbench as it is not synthesizable. Thanks to Zynq, you can test the module by the PS! Create your inputs on the microcontroller, pass them to your module on the PL side, and get the results from the module's output and verify it on the PS. Look at the diagram to understand the verification process better. Follow the steps bellow to easily perform the verification.

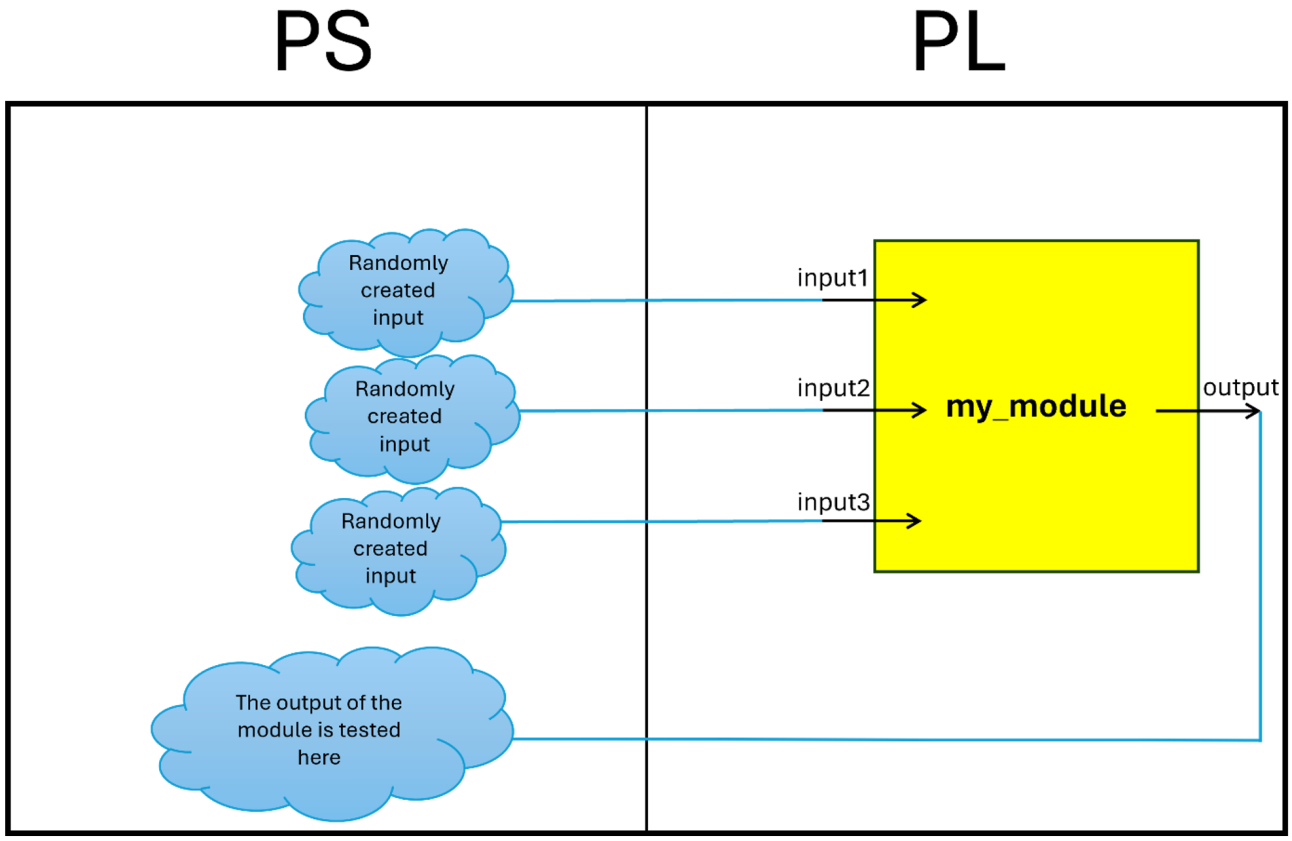

Figure 1.11: Block Diagram of PL module and its connection to PS inputs/outputs

First, describe a 32-bit combinational ALU able to execute ADD, SUB, NAND, and NOR functions on the two 32-bit inputs using Verilog HDL. Any carry or overflow should be ignored. There must be one OP input to customize the ALU, selecting which of the four operations is going to be performed.

Then, create another module that takes two 2-bit inputs A and B. This module should XOR A and B (bitwise) and have a 2-bit output, C.

After adding both of your modules to Vivado, create a block design to add the needed IP Cores such as the PS and some AXI GPIOs.

Configure the IP Cores and attach them properly. A diagram for what is expected is shown in figure 1. One of the 2-bit inputs of the XOR module should be connected to 2 of the PLs' keys, and the other input of the module must be attached to the OP output of the PS. The output of the XOR module, inputs the OP input of the ALU. The XOR module is there to test your verification method. As long as the two key inputs of the modules are active, the PS evaluating the logic should not report any errors, but when any of the keys go high, the PS must report an error. You need to use both two channels of AXI GPIO IP cores to lessen the resources used as much as possible.

After exporting the implemented hardware, open the SDK and start writing a software program

that creates random input vectors and their results to test the module.

After any error, the evaluation should stop for 2 seconds and the expected result versus the gotten result from our module must be shown via uart on your pc during the test, and the go on. At the end of evaluation, number of tests and the number of errors are to be shown on your PC.

Department of Electrical Engineering